Week 2 and 3 questions — exam-style answers

These are **full answers**, written the same way as THEORY-NOTES.md sections 29–36.
Cover the page. Write it yourself. Then check.

Questions: 02-Week2-and-Week3-Questions.pdf.

Easy

1. Who invented LeNet and what was its primary application?

Yann LeCun (and his team) invented LeNet (1998).

It was a very small CNN: conv → pool → fully connected. Its primary application was **reading handwritten digits**, especially **numbers on bank cheques**.

Paper line: *LeNet proved CNNs on digits.*

---

2. What dataset was LeNet trained on?

LeNet was trained on **MNIST** — small 28×28 images of handwritten digits 0–9.

That dataset was the proof that a CNN can learn from pixels better than older hand-made feature methods.

---

3. What was AlexNet's biggest achievement in 2012?

AlexNet **won the ImageNet competition by a huge margin**. After this, everyone started using deep CNNs. It showed that deep networks can work at **large scale**, not only on tiny digits.

That year is treated as the start of the deep-learning boom in computer vision.

---

4. Name two innovations introduced in AlexNet.

Any two of these (the notes call them **RDG** — ReLU, Dropout, GPU):

1. **ReLU** instead of sigmoid/tanh — training is faster and vanishing gradient is reduced.
2. **Dropout** — randomly switch off some neurons during training so the network cannot just memorise the data.
3. **GPU training** — made it possible to train such a big network on ImageNet.

Data augmentation is also a fair extra if they ask for more than two.

---

5. What activation function replaced sigmoid/tanh in AlexNet?

ReLU (Rectified Linear Unit).

ReLU(x) = max(0, x). Negative values become 0. Positive values stay. It adds non-linearity. It is faster than sigmoid and helps with vanishing gradients.

---

6. What does ReLU do to negative input values?

It turns them to **0**. Positive values stay as they are.

```
ReLU(x) = max(0, x)
```

Without an activation, many layers would act like just one layer. ReLU is the bend. It is also faster than sigmoid and helps vanishing gradients.

---

7. What is the purpose of dropout in training neural networks?

Dropout is regularisation. Its purpose is to stop the network from **memorising** (overfitting).

While training, randomly switch off some neurons (say 30%). They output 0 for that step. The network cannot depend on one "star" neuron. Everyone must learn. When you **test**, dropout is **off** — all neurons work.

Think of a team: random players sit out in practice, so the whole team knows the game.

Paper line: *Dropout is regularisation: random neurons are set to zero during training so the model does not overfit. At test time dropout is off.*

---

8. How many layers does VGG typically have?

VGG is typically **16 or 19 layers** (VGG-16, VGG-19).

The design is simple: it just stacks many small **3×3 filters**. Memory trick from notes: *VGG = small filters, big depth*.

---

9. What filter size did VGG standardize?

3×3, stacked many times.

Two 3×3 layers cover about the same area as one 5×5 filter, but with **fewer parameters** and **more non-linearity** (more ReLUs).

---

10. What problem does ResNet solve?

If you keep adding layers, training breaks. **Gradients vanish** and accuracy can actually get **worse**. That drop is called **degradation**.

ResNet's skip connections solved vanishing gradients so **very deep nets** (50, 101, 152 layers) could train.

---

11. What is the key innovation in ResNet?

Skip (residual) connections.

The input is copied forward and added to the layer's output:

output = F(x) + x

F(x) is what the layer learns (the "change" to make). x is the original input passed along. Gradients can flow easily through the skip path.

Paper line: *ResNet's skip connections solved vanishing gradients so very deep nets could train.*

---

12. What does EfficientNet's compound scaling balance?

Earlier models improved by only going deeper, or only wider, or only using bigger images. That wastes computation.

Compound scaling increases **depth, width and image resolution together**, using one scaling factor.

Paper line: *EfficientNet's compound scaling balances depth, width and resolution.*

---

13. Which CNN architecture enabled training of networks with 152 layers?

ResNet (ResNet-152).

The skip path `F(x) + x` lets the gradient travel back, so 50, 101 and 152 layers can be trained. Before ResNet, stacking that deep made accuracy worse.

---

14. Which architecture is lightweight yet accurate for mobile vision apps?

EfficientNet (the smaller versions).

It gets high accuracy with **fewer parameters**. It works well on phones and small devices because it does not waste compute by scaling only one thing.

---

15. What does object detection aim to identify in an image?

Detection finds **what** is in the image and **where**.

Classification says “there is a dog”. Detection draws a **rectangle** around each dog and names it: **box + class** for each object. Example: “cat here, dog there”.

Paper line: *Detection gives a box and class for each object.*

---

16. What does R-CNN stand for?

Region-based Convolutional Neural Network.

Name = Region-based CNN. It guesses many possible boxes (regions), then runs a CNN on each box.

---

17. Why is YOLO considered fast?

YOLO means **You Only Look Once**.

It looks at the **whole image once**. In that one forward pass it predicts all boxes and all classes together. There is no “thousands of CNN runs on separate crops”.

That is why it is used for **real-time** work: driving, sports, live CCTV.

---

18. What does SSD stand for?

Single Shot Detector / **Single Shot MultiBox Detector**.

It is also one shot (not two stages). It looks at **several feature maps of different sizes**, so it can catch small and large objects.

---

19. What is semantic segmentation?

Semantic segmentation labels every pixel with a class. Not one label for the photo. Not a box. The actual shape.

Example: this pixel is road, this is sky, this is car. **Two cars get the same label** — both are just “car”. It does not split car-1 and car-2. That extra split is **instance** segmentation.

Paper line: *Semantic segmentation labels every pixel (same class = same colour).*

20. What is the role of the encoder in U-Net?

Draw U-Net in your head:

```
Encoder (down):  picture gets smaller → understands WHAT it is
Decoder (up):    picture grows back   → understands WHERE it is
Skip connections: copy encoder details into decoder → sharp edges
```

The **encoder is the down path**. It compresses the picture for **meaning / context** ("this looks like tissue"). The decoder later expands for place. Skips bring back the original edges.

Paper line: *The encoder captures context.*

Medium

1. How did LeNet prove CNNs were better than traditional ML methods?

Traditional methods extracted features **by hand**, then gave a vector to a classifier (kNN, SVM).

LeNet was a small CNN trained **end-to-end** on **MNIST** pixels: conv → pool → fully connected. It **learned the filters from the image itself**. On handwritten digits (cheques) this worked better than the older pipeline.

Problem it solved: it was a **proof of concept** that CNNs work on images.

---

2. Why was GPU training critical for AlexNet's success?

AlexNet is big: 5 conv layers + 3 fully connected layers, trained on **ImageNet** (real photos, not 28×28 digits).

On a CPU that training would take far too long. **GPUs** do many multiply-adds at once, so they could train such a network in a realistic time. Without the GPU, the 2012 result would not have happened at that scale.

Memory trick: AlexNet = **RDG** — ReLU, Dropout, **GPU**.

---

3. Compare accuracy before and after AlexNet on ImageNet.

Before 2012, the best ImageNet methods had much **higher error** (around **26%** top-5 error). They were not deep CNNs at this scale.

AlexNet dropped that error to about **15%**. That is a large jump. After this, everyone started using deep CNNs.

You do not need the exact percent if you forget it. Write: *AlexNet won ImageNet by a huge margin and started the deep-learning boom.*

---

4. Why does stacking small filters (like 3×3 in VGG) improve performance?

VGG's idea: go deeper with a **simple** design — many **3×3** filters.

Two 3×3 layers cover about the **same area** as one 5×5 filter, but with **fewer parameters** and **more non-linearity** (two ReLUs instead of one). So you get a bigger field of view, cheaper weights, and more "bends".

Problem it solved: how to go deeper without a messy design. **Limitation:** still very heavy (huge number of parameters).

---

5. How do residual connections help gradient flow in ResNet?

In a plain deep stack, backprop multiplies many small numbers. The gradient **vanishes**. Early layers stop learning.

A skip copies **x** forward: **output = F(x) + x**. The extra `+ x` is a **highway**. The gradient can travel **back through the add** without dying in many multiplies. Deep layers still get a signal.

That is why 50 / 101 / 152 layers can train.

Paper line: *In ResNet they help gradient flow so deep nets can train.*

---

6. What is the difference between direct mappings and residuals?

Direct mapping: the layer must learn the full output **H(x)** from scratch. If the right answer is almost the same as the input, this is hard. The layer may mess up a signal that was already good.

Residual: the layer only learns the **change F(x) = H(x) − x**, then adds the input: **H(x) = F(x) + x**. If nothing extra is needed, **F can be near zero** and the skip just copies x. Easier to learn. This is why extra layers do not have to make the net worse.

---

7. Why is EfficientNet more efficient than simply making networks deeper or wider?

If you **only** add layers, or **only** add channels, or **only** use a bigger image, you waste computation. One direction grows, the others stay unbalanced.

Compound scaling increases **depth, width and resolution together** with one factor. You get high accuracy with **fewer parameters**. That is why it fits phones.

Paper line: *EfficientNet's compound scaling balances depth, width and resolution.*

---

8. Give one real-world application of ResNet.

ResNet is very popular for **transfer learning** and **medical imaging**.

A ResNet already trained on ImageNet has learned edges, textures and shapes. For hospital images you freeze early layers and train the last layers on your scans. ResNet is also the usual **backbone** inside Faster R-CNN and Mask R-CNN.

Write one: *medical scan classification using a pre-trained ResNet*, or *backbone for a detector*.

---

9. Give one real-world application of EfficientNet.

Phone cameras and other **on-device** vision apps — classify a plant or a product on a mobile phone without a huge model.

EfficientNet gives high accuracy with fewer parameters, so it runs on small devices.

---

10. Why is R-CNN slow compared to YOLO?

R-CNN: guess many possible boxes (classically about 2000), then **run a CNN on each box**. Thousands of CNN runs. Accurate, **very slow**. Fine for a doctor looking at a scan later. Bad for a car that must decide now.

YOLO: looks at the **whole image once**. One forward pass, all boxes and classes together.

That is the speed gap.

---

11. How does YOLO treat detection differently from R-CNN?

R-CNN is two ideas in a row: **propose regions, then classify** each region. Faster R-CNN still has two stages: first an **RPN** proposes, then classify.

YOLO treats detection as **one regression problem**. It looks at the whole image **once** and predicts all boxes and all classes together. No separate “crop then CNN” loop.

Same job (box + class). Different process.

---

12. Why is SSD better at detecting objects of different sizes?

SSD is also **single-shot**, but it looks at **several feature maps of different sizes**.

Shallow / finer maps see **small** objects. Deeper / coarser maps see **large** objects. That multi-scale idea is why SSD handles mixed sizes better than a detector that only looks at one map.

It sits in the **middle** of speed and accuracy. Common on phones.

---

13. Which detection architecture would you choose for medical imaging and why?

Choose the **R-CNN family** (Faster R-CNN). If you also need the **exact shape** of a tumour, choose **Mask R-CNN**.

Hospital work needs **accuracy**, not 60 frames per second. A doctor can wait. R-CNN is accurate but slow — that is acceptable here.

They love this choice: *Medical / tumour → need accuracy → R-CNN family.*

---

14. Which detection architecture would you choose for autonomous driving and why?

Choose **YOLO** (or SSD).

A car must decide **now** — a pedestrian in two seconds is too late. YOLO looks once and is **real-time**. Older YOLO can miss **tiny** far objects; still the usual driving / live CCTV choice.

They love this choice: *Self-driving / live camera → need speed → YOLO (or SSD).*

---

15. What challenge does semantic segmentation face compared to classification?

Classification gives **one label** for the whole photo: “this is a cat”.

Semantic segmentation must name **every pixel** and follow the **exact outline**. You need both **what** and **where**. You collect far more labels (every pixel). Memory is higher. Edges are easy to get **coarse** after the image is downsampled.

That is why FCN masks look rough, and why U-Net adds skips for sharp borders.

---

16. How does FCN differ from traditional CNNs in its output?

Old CNNs ended with **fully connected** layers. Those layers **flatten** the picture, so “where” is lost. Output is one class vector.

FCN removes those dense layers and uses only convolutions. Output is a **map** the same idea as the image: each pixel has a class.

Plus: first big idea for pixel labels. **Minus:** the mask is **rough**. Edges look fuzzy.

---

17. Why are FCN predictions often coarse?

The encoder **downsamples** a lot. Fine edges are lost. Upsampling cannot fully bring thin borders back. The tumour outline looks **blocky / fuzzy**.

U-Net was designed to fix this with **skip connections** that copy high-resolution features into the decoder.

---

18. How do skip connections in U-Net improve segmentation accuracy?

U-Net skips are **not** the same job as ResNet skips, even though the word is the same.

U-Net **copies encoder details into the decoder**. Compression had lost the original edges. The decoder now sees both **what** (deep context) and **where** (shallow sharp pixels). Borders of the organ become **sharp**.

Paper line: *In U-Net they copy high-resolution features to the decoder for sharp masks.*

---

19. What is the role of the decoder in U-Net?

The **decoder is the up path**. The picture **grows back**. It understands **WHERE** the pixels are and paints the **pixel labels**.

Alone it would be blurry. **Skips** from the encoder bring the original edges so the outline is precise.

Paper line: *The decoder recovers location, skips bring fine detail.*

---

20. Why is U-Net widely used in medical imaging?

Medical photos need the **exact boundary**, not a box. A box around a tumour includes healthy tissue.

U-Net is an encoder-decoder with skip connections: the encoder captures context, the decoder recovers location, skips bring fine detail. It also works with **relatively few labeled scans**.

Paper line: *This is why U-Net is standard in medical imaging.*

Hard

1. Explain how dropout improves generalization with an example.

Without dropout, the net can memorise one lucky path: “if this star neuron fires, it is a cat.” That works on the training set and **fails on a new photo**. That is overfitting.

Dropout randomly sets some neurons to zero during training. The network cannot depend on one neuron. Many small groups must learn the same idea.

Example: a face network that overfits to one indoor lamp. Dropout forces it to use eyes + nose + outline together. A new photo in sunlight still works.

At **test** time dropout is **off**. All neurons work.

---

2. Why did AlexNet's use of ReLU help reduce the vanishing gradient problem?

Sigmoid / tanh flatten at the ends. There the gradient is almost **0**. In a deep net, backprop multiplies many of those tiny numbers. Early layers stop learning. That is vanishing gradient.

ReLU has gradient **1** for every positive input. The signal stays alive through many layers. Training is also **faster**.

That is why ReLU replaced sigmoid/tanh in AlexNet and stayed.

---

3. How does compound scaling in EfficientNet systematically balance depth, width, and resolution?

The problem: earlier models improved by only going deeper, or only wider, or only using bigger images. That wastes computation.

EfficientNet starts from a good small baseline. Then it uses **one scaling factor**. **Depth, width and image resolution increase together** by a fixed rule, not by guessing "add 20 layers" or "double channels".

Result: high accuracy with fewer parameters. Works on phones.

---

4. Why do residual connections allow training of very deep networks without degradation?

Degradation: a very deep plain net can get **worse**, not better.

With **output = F(x) + x**, if extra layers are useless they can learn **F ≈ 0**. The skip copies **x**. The deep net is at least as good as the shallow one. Gradients also flow through the skip, so vanishing gradient is reduced.

That is how 50 / 101 / 152 layers train. ResNet became the default backbone.

---

5. Compare the trade-off between accuracy and speed in R-CNN, YOLO, and SSD.

Write this as a real paragraph, not only a table.

R-CNN family: classifies region proposals with a CNN. **Accurate but slow** (classic R-CNN is thousands of CNN runs). Faster R-CNN adds an RPN but is still two-stage. Choose when **accuracy matters more than speed** — medical / tumour.

YOLO: predicts boxes and classes in **one forward pass**. **Real-time**. Weaker on **tiny** objects. Choose when **real-time is required** — driving, live CCTV.

SSD: also single-shot. Uses **multi-scale feature maps**, so mixed sizes are better than early YOLO. Middle of speed and accuracy. Common on phones.

Paper line: *Choose R-CNN when accuracy matters more than speed; YOLO/SSD when real-time is required.*

---

6. How would you adapt YOLO for detecting very small objects?

YOLO's known minus: **weak on tiny objects** (a far person can be missed). The grid cell is too coarse, and the object is a few pixels.

Ways to adapt (write two or three):

- Detect at a **higher input resolution** so a small object covers more cells.
- Use a **finer grid** / an extra **small-object head** on a **shallower feature map** (the same multi-scale idea as SSD).
- Add more **small-box anchors**.
- **Tile / crop** the image (zoom pieces) and run YOLO on each tile.

You are giving YOLO a closer look at small things, which costs some speed.

---

7. Why are skip connections critical in both ResNet and U-Net, though used differently?

The shortcut is the same idea: **copy input forward**. The *reason* is different.

ResNet skip: so a **very deep** net can still train. **output = F(x) + x**. Helps the **gradient** flow back (vanishing gradient). Used in classification / as a backbone.

U-Net skip: so **edges / small details** are not lost. Copies **high-resolution features** from encoder to decoder for **sharp masks**. Used in segmentation.

Paper line: *Skip connections copy input forward. In ResNet they help gradient flow so deep nets can train. In U-Net they copy high-resolution features to the decoder for sharp masks.*

---

8. How does the encoder-decoder structure in U-Net ensure both context and precision?

Encoder (down): picture gets smaller → understands **WHAT** it is. That is **context** (this region looks like a kidney).

Decoder (up): picture grows back → understands **WHERE** it is. That is **place** (these pixels).

Skip connections: copy encoder details into the decoder → **sharp edges**. Compression had thrown those edges away. Skips bring them back.

Together: context + location + fine detail. That is why the tumour outline is usable.

---

9. If you were designing a segmentation model for satellite imagery, which architecture would you choose and why?

Choose **U-Net** (or an FCN with skips).

A satellite map needs **every pixel**: road, water, field, building. A box is not enough. U-Net keeps **context** (a river) and **thin lines** (a road) because of encoder–decoder + skips.

If the question is "each building separately" (building-1 vs building-2), that is **instance** segmentation — then **Mask R-CNN**. For the usual land-cover map, U-Net is the answer.

---

10. How did the evolution from LeNet → AlexNet → VGG → ResNet → EfficientNet shape the foundation for Vision Transformers?

Each architecture fixed a problem from the one before. Story in one line from the notes: *LeNet proved CNNs work → AlexNet made them big and fast → VGG made them deep and simple → ResNet made very deep nets trainable → EfficientNet made them efficient.*

That line taught the field: learn filters from data, train at ImageNet scale with **ReLU, dropout, GPU**, go deeper, then **skip** so depth does not die, then **scale efficiently**.

ViT then said: split the image into **patches** and use **attention**, not only a small CNN window. Attention lets far parts talk in **one step**. ViT still uses the same lessons: lots of data, residual-style blocks, careful scaling. CNNs built the road. ViT walks it with attention.

Paper line: *Attention uses Query, Key and Value… ViT splits an image into patches, treats them as tokens, and runs a transformer encoder… This is how ViT sees global context that local CNN filters miss.*

Mask R-CNN

1. What key limitation of traditional object detection does Mask R-CNN address?

A **box is not the shape**. A box around a tumour includes healthy tissue. Two people standing together: **boxes overlap**; you cannot tell who is who by the rectangle.

Mask R-CNN gives a **pixel-level mask** for **each** object — the exact shape, even if objects overlap.

---

2. How does Mask R-CNN extend Faster R-CNN?

Mask R-CNN (2017) = Faster R-CNN + one extra branch that draws a black-and-white **mask** for each object.

Same backbone, same **RPN**, same box and class. The new head outputs the exact pixels.

Paper line: *Mask R-CNN extends Faster R-CNN with a mask branch.*

---

3. What role does the Region Proposal Network (RPN) play in Mask R-CNN?

The **RPN** looks at the feature map and **proposes regions** that might contain an object.

After those proposals, **RoI Align** extracts aligned features. Then the heads output box, class and mask.

Faster R-CNN added the RPN so you do not run a CNN on thousands of random crops. Mask R-CNN keeps that RPN.

---

4. Why was RoI Align introduced, and how does it improve accuracy compared to RoI Pooling?

RoI Pool rounded coordinates to whole pixels. The cut feature was a bit **off**. The mask sat next to the object, not on it.

RoI Align does **not** round. It uses interpolation so the features **line up** with the box. The mask sits on the object.

This matters for **pixels**. A box can survive a small shift. A mask cannot.

---

5. What are the three outputs produced by Mask R-CNN for each detected object?

For each object:

1. **Box** — where
2. **Class** — what
3. **Mask** — the exact pixels

Paper line: *For each object it outputs a box, a class and a pixel-level mask.*

---

6. Why are pixel-level masks more useful than bounding boxes in medical imaging?

A box around a tumour includes **healthy tissue**. A **mask follows the tumour edge**. You get true **shape**, true **area / volume**, and you can separate **overlapping organs**.

That is what a doctor measures when they ask “how big is it?” and “what is the outline?”

---

7. Give two real-world applications of Mask R-CNN outside of medical imaging.

Write two:

1. **Self-driving / crowds** — separate each pedestrian or vehicle, even when they overlap.
2. **Robot picking** — grip the exact object, not a loose box of background.

(Video effects, satellite buildings, and retail checkout are also fine.)

---

8. How does Mask R-CNN handle overlapping objects in crowded scenes?

This is **instance segmentation**. Each object gets **its own mask**.

Two people in one messy box still have **two outlines**. The mask branch draws each person separately, so the overlap is not treated as one blob.

Sequence modeling for video (CNN + LSTM)

9. Why can't CNNs alone capture temporal dependencies in video data?

Purpose: understand **video**, not one photo. Video is photos **in order**.

A CNN sees **what** is in **one frame** (a hand, a ball). It does not remember "the hand was up, now it is down". **Time order is missing**.

One photo of a hand is not a "wave". A wave is the hand moving **across frames**.

Paper line: *A CNN sees what is in each frame (spatial features) but not time.*

---

10. What specific problem do LSTMs solve compared to vanilla RNNs?

An **RNN** has a small memory of recent steps. On a **long** video that memory fades. This is **vanishing gradient**. Early frames stop teaching the net.

LSTM (Long Short-Term Memory) is an RNN with **gates** so it can remember longer. It can also handle short and long videos.

Paper line: *An RNN models sequence but forgets long videos (vanishing gradient). LSTM uses input, forget and output gates to keep long-term memory.*

---

11. Explain the role of the input, forget, and output gates in LSTMs.

LSTM uses three gates:

- **Forget gate** — what old information to throw
- **Input gate** — what new information to keep
- **Output gate** — what to send to the next step

Think: erase / write / read. That is how the cell state stays useful across many frames.

---

12. In a CNN + LSTM pipeline, what does each component contribute to video understanding?

Pipeline from the notes: CNN per frame → list of feature vectors → LSTM → action class (clap, throw, walk).

- **CNN** on each frame → **what** is in that picture (a hand, a ball). Spatial features.
- **LSTM** on the sequence of those CNN results → **how it changes** (the wave, the throw). Time.

Paper line: *CNN + LSTM is used for action and gesture recognition.*

---

13. Provide an example of an application where CNN + LSTM is essential.

Gesture recognition, **action recognition** in video, lip reading, or video captioning.

A still photo is not enough. You need the **order of frames**. Example: tell a “wave” from a “stop” hand.

---

14. How does sequence modeling enable gesture recognition compared to static image classification?

Static classification: one photo → one label. Frame 1 of a wave looks like “hand on the left”. That is not the gesture.

A **wave is a sequence**. Frame 1 = hand left, later frames = hand right. The **LSTM** reads the CNN features **in order** and sees the path. That is sequence modeling. Classification of one image cannot do this.

RNNs vs LSTMs

15. What is the vanishing gradient problem, and why does it affect RNNs more than LSTMs?

When you train through many time steps, backprop **multiplies many small numbers**. The gradient goes toward **0**. Early frames stop learning.

A **vanilla RNN** has no special highway. On a long video the memory fades. That is why vanishing gradient hits RNNs hard.

An **LSTM** has a **cell state** and **gates**. The gradient can travel farther along that cell. LSTM keeps long-term memory.

---

16. Compare RNNs and LSTMs in terms of memory and best use cases.

RNN: short memory. Best for **short** sequences — a few steps, a simple series. Cheap.

LSTM: gated cell, **long** memory. Best for **video, speech, long text**, and variable-length clips. Heavier than a vanilla RNN, but it does not forget the start of a long video as quickly.

---

17. Why are LSTMs better suited for variable-length sequences like video?

A clip can be 20 frames or 200. LSTM reads **one step at a time** and keeps a **state**. Length can change. Gates decide what to keep across a long clip.

A model that expects a fixed number of frames would break. LSTM does not need a fixed length.

---

18. Give an example of a task where RNNs might still be sufficient instead of LSTMs.

A **short** sequence where long memory is not needed: 3–5 frames, or a tiny time series (a few sensor steps). LSTM would be extra cost.

Example: classify a 4-frame blink, or a very short digit sequence.

Vision Transformers and attention

19. What is the purpose of the Query, Key, and Value in the attention mechanism?

Purpose: a CNN looks at a **small window**. Attention lets far parts **talk in one step**.

Three names (from language models, used in ViT):

- **Query (Q)** — what I am looking for
- **Key (K)** — what each piece is
- **Value (V)** — the information inside that piece

Compare Q with K → get importance scores → mix the Values with those scores. Related pieces get high weight.

Paper line: *Attention uses Query, Key and Value to decide which parts of the input matter and captures long-range relations.*

20. Why does splitting an image into patches help Vision Transformers capture global context?

A transformer cannot easily eat a raw 224×224 grid as one blob.

ViT steps:

1. Cut the image into **patches** (e.g. 16×16). Flatten each patch. Treat it like a **word**.
2. Send this list of patches into a **Transformer** (attention).
3. A special **class token** summarises the whole image for the final label.

Patches keep **local** detail. Attention across patches is **global**: patch 1 can look at patch 196 in **one step**. A CNN needs many layers to see that far. That is how ViT sees global context that local CNN filters miss.

ViT usually needs **a lot of data**.

After this file: write the Mask R-CNN three outputs and the ResNet vs U-Net skip paragraph from memory. Those two come almost every paper.